

Testing & Validation Plan

Automated Web Vulnerability Scanner

Digital Egypt Pioneers — DEPI Team 2

1. Overview

This document defines the Testing & Validation strategy for the Automated Web Vulnerability Scanner project. It covers unit-level test cases for each scanner module, integration testing across all components, and acceptance criteria for the final demonstration.

Three vulnerability scanners are covered:

1. Reflected XSS Scanner (xss_scanner.py)
2. Sensitive File Disclosure Scanner (sfd_scanner.py)
3. Broken Access Control / IDOR Scanner

2. Reflected XSS — Test Cases

2.1 Scanner Behavior

The XSS scanner performs the following steps:

4. Recon: Discovers URL parameters by brute-forcing common names (name, q, search, query, etc.)
5. Injection: Injects 5 XSS payloads into each discovered parameter.
6. Detection: Uses Regex patterns to check if the payload was reflected unencoded in the response.
7. Output: Returns a dictionary with vulnerability_name, is_vulnerable, payload_used, and technical_steps.

2.2 Payloads Used

Payload #	Payload String	Technique
1	<script>alert(1)</script>	Basic script injection
2	"><script>alert(1)</script>	Attribute breakout
3		Event handler injection
4	<svg onload=alert(1)>	SVG onload event
5	'><script>alert(1)</script>	Single-quote breakout

2.3 Test Cases

Test ID	Test Name	Input / Payload	Expected Result	Pass Condition
TC-XSS-01	Basic Script Injection	<script>alert(1)</script>	VULNERABLE	Payload reflected unencoded in HTML
TC-XSS-02	Attribute Injection	"><script>alert(1)</script>	VULNERABLE	Breaks out of attribute, executes JS
TC-XSS-03	Image onerror		VULNERABLE	onerror handler triggers alert
TC-XSS-04	SVG onload	<svg onload=alert(1)>	VULNERABLE	SVG onload event executes JS
TC-XSS-05	Single Quote Bypass	'><script>alert(1)</script>	VULNERABLE	Escapes single-quote attribute
TC-XSS-06	Safe Input (plain text)	hello	SAFE	Plain text reflected safely
TC-XSS-07	Encoded Payload	<script>alert(1)</script>	SAFE	Properly encoded — not executable
TC-XSS-08	No Parameters	http://site.com/page	SAFE	No input params — recon returns empty

3. Sensitive File Disclosure — Test Cases

3.1 Scanner Behavior

The SFD scanner probes a target URL with 20 built-in sensitive paths (or a custom wordlist). For each path, it sends an HTTP GET request and flags exposure when the response returns HTTP 200 with a non-empty body.

3.2 Paths Tested (sample)

Path	Sensitivity Reason
/.env	Contains API keys, DB credentials
/.git/config	Exposes repository structure
/config.php	PHP config with DB passwords
/phpinfo.php	Reveals server internals
/backup.zip	Full application backup
/wp-config.php	WordPress DB credentials

3.3 Test Cases

Test ID	Test Name	Request	Expected Result	Pass Condition
TC-SFD-01	/.env exposure	GET /.env	VULNERABLE	HTTP 200 + content > 0 bytes
TC-SFD-02	/.git/config exposure	GET /.git/config	VULNERABLE	Git config accessible publicly
TC-SFD-03	/config.php exposure	GET /config.php	VULNERABLE	PHP config returned with 200
TC-SFD-04	/phpinfo.php	GET /phpinfo.php	VULNERABLE	PHP info page publicly exposed
TC-SFD-05	/backup.zip	GET /backup.zip	VULNERABLE	Backup archive accessible
TC-SFD-06	Non-existent file	GET /notexist.xyz	SAFE	HTTP 404 — file not found
TC-SFD-07	Redirect on sensitive path	GET /admin	SAFE	HTTP 302 redirect — not exposed
TC-SFD-08	Empty response body	GET /empty.env	SAFE	HTTP 200 but content = 0 bytes

4. Broken Access Control / IDOR — Test Cases

4.1 Scanner Behavior

The IDOR scanner tests whether the application enforces proper authorization checks on object references. It attempts to access resources belonging to other users by manipulating ID parameters in requests.

4.2 Test Cases

Test ID	Test Name	Request	Expected Result	Pass Condition
TC-IDOR-01	Access own resource	GET /user/profile?id=1 (own)	SAFE	Authorized user accesses own data
TC-IDOR-02	Access other user resource	GET /user/profile?id=2 (other)	VULNERABLE	HTTP 200 returns another user's data
TC-IDOR-03	Sequential ID enumeration	GET /orders?id=100,101,102	VULNERABLE	All IDs return data without auth check
TC-IDOR-04	Modify other user data	POST /user/update?id=2	VULNERABLE	Changes another user's record
TC-IDOR-05	Access admin resource	GET /admin/dashboard?id=1	VULNERABLE	Non-admin accesses admin resource
TC-IDOR-06	Invalid ID format	GET /user/profile?id=abc	SAFE	HTTP 400 or 404 — invalid format
TC-IDOR-07	Access with valid auth	GET /user/profile?id=1 + token	SAFE	Proper auth check — access granted
TC-IDOR-08	Access without auth	GET /user/profile?id=1 (no token)	SAFE	HTTP 401 — unauthorized

5. Integration Testing

Integration tests verify that all scanner modules work together correctly through the shared BaseScanner interface, and that results flow correctly to the reporter and frontend.

5.1 Shared Dictionary Format

All scanners must return a dictionary in the following structure:

```
{ "vulnerability_name": str, "is_vulnerable": bool, "payload_used": str,
  "technical_steps": [ {"action": str, "details": str} ] }
```

5.2 Integration Test Cases

Test ID	Test Scenario	Action	Expected Result	Pass Condition
TC-INT-01	Full scan — vulnerable app	Run all 3 scanners on DVWA	PASS	All 3 scanners detect vulnerabilities
TC-INT-02	Full scan — clean app	Run all 3 scanners on safe site	PASS	All 3 scanners return is_vulnerable: False
TC-INT-03	Dictionary format consistency	Check output format of each scanner	PASS	All return same dictionary structure
TC-INT-04	Report generation	Feed scan results to reporter	PASS	Markdown report generated correctly
TC-INT-05	Frontend display	Pass results to visual timeline	PASS	Timeline renders all steps correctly

6. Acceptance Criteria

The following conditions must be met before the final demonstration:

- All 3 scanners successfully detect vulnerabilities on the test target (DVWA).
- All scanners return SAFE on a clean target.
- All scanner outputs match the agreed dictionary format.
- The reporter module generates a valid Markdown report from scanner output.
- The frontend visual timeline displays all technical_steps correctly.
- End-to-end demo runs without errors on the live presentation.

7. Sample Scanner Output

The following is a real output produced by the XSS Scanner when run against DVWA (Reflected XSS page):

7.1 Vulnerable Target — Output

```
{
  "vulnerability_name": "Reflected XSS",
  "is_vulnerable": true,
  "payload_used": "<script>alert(1)</script>",
  "technical_steps": [
    { "action": "Recon", "details": "Discovered parameter: name" },
    { "action": "Parameter Tested", "details": "name" },
    { "action": "Payload Sent", "details": "<script>alert(1)</script>" },
    { "action": "Reflection Found", "details": "Payload reflected in response" },
    { "action": "XSS Pattern Detected", "details": "<script.*?>" },
    { "action": "URL Tested", "details": "" }
  ]
  "http://localhost/DVWA/vulnerabilities/xss_r/?name=..." }
}
```

Result: is_vulnerable = true — Scanner correctly identified the Reflected XSS vulnerability on DVWA.

8. Tools & Environment

Tool / Environment	Purpose
DVWA (Damn Vulnerable Web App)	Main vulnerable target for all scanner testing
Python 3.x + requests + re	Scanner execution environment
pytest (optional)	Automated unit test runner
GitHub	Version control & code review
VSCoDe	Development environment